

UNIT 5

TRANSPORT LAYER

LH - 7HRS

5.1 Functions of Transport Layer

1. Service Point Addressing: Transport Layer header includes service point address which is port address. This layer gets the message to the correct process on the computer unlike Network Layer, which gets each packet to the correct computer.

2. Segmentation and Reassembling: A message is divided into segments; each segment contains sequence number, which enables this layer in reassembling and replaces packets which were lost in transmission.

3. Connection Control: It includes 2 types:

Connection Oriented- Before delivering packets, connection is made with transport layer at the destination machine.

Connectionless: Each segment is considered as an independent packet and delivered to the transport layer at the destination machine.

4. Flow control: In this layer, flow control is performed end to end.

5. Error control: Error control is performed end to end in this layer to ensure that the complete message arrives at the receiving transport layer without any error. Error correction is done through retransmission.

5.2 Elements/Responsibilities of Transport Protocols

1. Process to process delivery:

Transport Layer requires a port number to correctly deliver the segments of data to the correct process among the multiple processes running on a particular host. A port number is a 16 bit address used to identify any client-server program uniquely.

2. End-to-end Connection between hosts:

The transport layer is also responsible for creating the end-to-end connection between hosts for which it mainly uses TCP and UDP. TCP is a connection-orientated protocol which uses a handshake protocol to establish a robust connection between two end- hosts. UDP, on the other hand, is a connectionless protocol which ensures best-effort delivery.

3. Multiplexing and Demultiplexing:

Multiplexing allows simultaneous use of different applications over a network which is running on a host. The transport layer provides this mechanism which enables us to send packet streams from various applications simultaneously over a network. Similarly, Demultiplexing is required at the receiver side to obtain the data coming from various processes.

4. Congestion Control:

Transport layer uses open loop congestion control to prevent the congestion and closed loop congestion control to remove the congestion in a network once it occurs.

5. Data integrity and Error correction:

The information that is coming from the application layer might have errors and those are checked by the transport layer through the calculation of checksums. It analyses whether the received information is corrupted or not and through NACK and ACK services, it notifies the sender if the information has arrived or not and ensures data integrity.

6. Flow control:

The transport layer provides a flow control mechanism between the adjacent layers of the TCP/IP model. TCP also prevents data loss due to a fast sender and slow receiver by imposing some flow control techniques. It uses the method of sliding window protocol which is accomplished by the receiver by sending a window back to the sender informing the size of data it can receive.

5.3 User Datagram Protocol (UDP)

- User Datagram Protocol (UDP) is a Transport Layer protocol referred as UDP/IP suite.

It is unreliable and connectionless protocol, So there is no need to establish connection prior to data transfer.

- For the realtime services like computer gaming, voice or video communication, live conferences; we need UDP.
-

Since high performance is needed, UDP permits packets to be dropped instead of processing delayed packets.

No error checking in UDP, so it also save bandwidth.

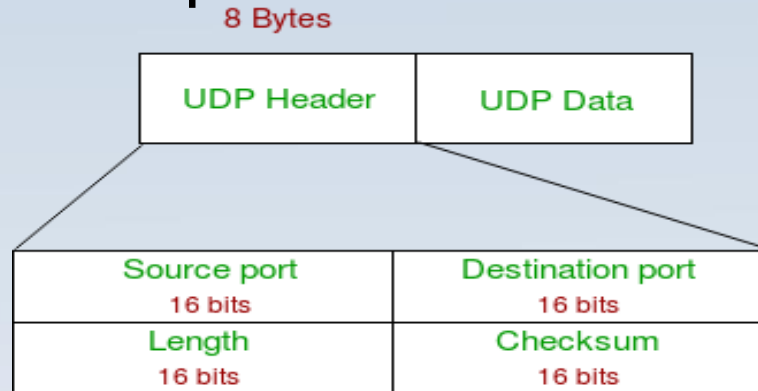
- User Datagram Protocol (UDP) is more efficient in terms of both latency and bandwidth.

UDP Header:

UDP header is 8-bytes fixed and simple header.

First 8 Bytes contains all necessary header information and remaining part consist of data.

UDP port number fields are each 16 bits long, therefore range for port numbers defined from 0 to 65535; port number 0 is reserved. Port numbers help to distinguish different user requests or process.



1. Source Port : Source Port is 2 byte long field used to identify port number of source.
2. Destination Port : It is 2 byte long field, used to identify the port number of destination.
3. Length : It is 2 bytes long field which is the length of UDP including header and the data.
4. Checksum : Checksum is 2 bytes long field for error control.

Notes - Unlike TCP, Checksum calculation is not mandatory in UDP. No Error control or flow control is provided by UDP. Hence UDP depends on IP and ICMP for error reporting.

UDP Operations:

The basic steps for transmission using UDP are:

- Higher-Layer Data Transfer: An application sends a message to the UDP software.
- UDP Message Encapsulation: The higher-layer message is encapsulated into the Data field of a UDP message. The headers of the UDP message are filled in, including the Source Port of the application that sent the data to UDP, and the Destination Port of the intended recipient. The checksum value may also be calculated.
- Transfer Message To IP: The UDP message is passed to IP for transmission.

On reception at the destination device this short procedure is reversed.

Applications of UDP:

- Used for simple request response communication when size of data is less and hence there is lesser concern about flow and error control.
- It is suitable protocol for multicasting as UDP supports packet switching.
- UDP is used for some routing update protocols like RIP(Routing Information Protocol).
- Normally used for real time applications which can not tolerate uneven delays between sections of a received message.
- UDP takes datagram from Network Layer, attach its header and send it to the user. So, it works fast.
- Following implementations uses UDP as a transport layer protocol:
 - DNS (Domain Name Service)
 - DHCP

5.4 Transmission Control Protocol (TCP)

- The Transmission Control Protocol is the most common transport layer protocol.
- It works together with IP and provides a reliable transport service between processes using the network layer service provided by the IP protocol.

The various services provided by the TCP to the application layer are as follows:

1. Process-to-Process Communication:

- TCP provides process to process communication, i.e, the transfer of data takes place between individual processes executing on end systems. This is done using port numbers or port addresses.

2. Full duplex service:

This means that the communication can take place in both directions at the same time.

3. Connection oriented service:

Unlike UDP, TCP provides connection oriented service. It defines 3 different phases:

- Connection establishment
- Data transfer
- Connection termination

4. Reliability:

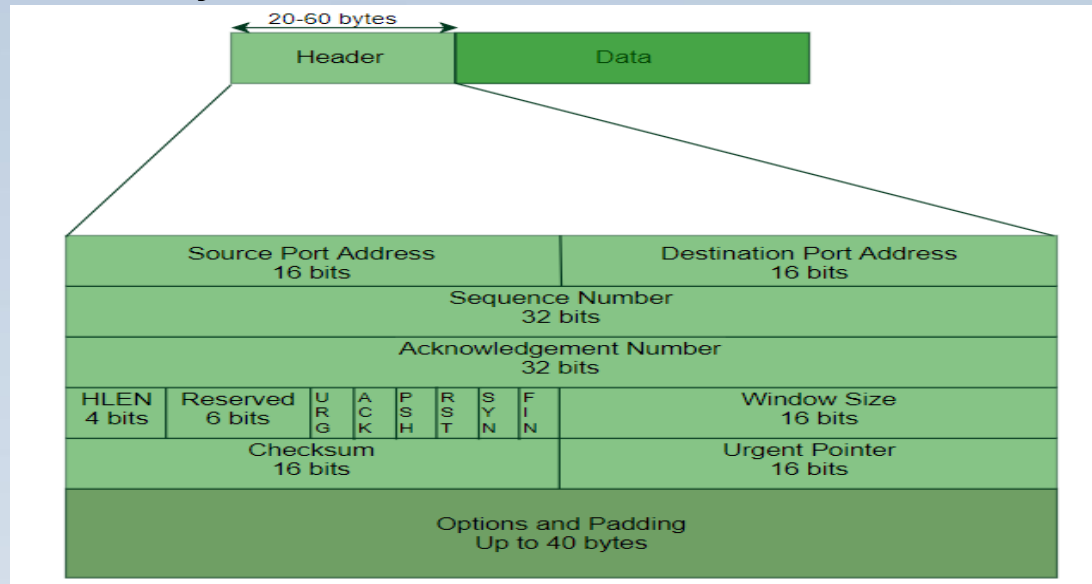
TCP is reliable as it uses checksum for error detection, attempts to recover lost or corrupted packets by re-transmission, acknowledgment policy and timers. It uses features like byte number and sequence number and acknowledgment number so as to ensure reliability.

5. Multiplexing:

TCP does multiplexing and de-multiplexing at the sender and receiver ends respectively as a number of logical connections can be established between port numbers over a physical connection.

TCP Segment Header Format

TCP segment consists of data bytes to be sent and a header that is added to the data by TCP as shown:



The header of a TCP segment can range from 20-60 bytes. 40 bytes are for options. If there are no options, header is of 20 bytes else it can be of upmost 60 bytes.

Header fields:

- Source Port Address -

16 bit field that holds the port address of the application that is sending the data segment.

- Destination Port Address -

16 bit field that holds the port address of the application in the host that is receiving the data segment.

- Sequence Number -

32 bit field that holds the sequence number. It is used to reassemble the message at the receiving end if the segments are received out of order.

- Acknowledgement Number -

32 bit field that holds the acknowledgement number. It contains sequence number of the data byte that receiver expects to receive next from the sender.

- Header Length (HLEN) -

This is a 4 bit field that indicates the length of the TCP header by number of 4-byte words in the header, i.e, if the header is of 20 bytes(min. length of TCP header), then this field will hold 5 (because $5 \times 4 = 20$) and the maximum length: 60 bytes, then it'll hold the value 15(because $15 \times 4 = 60$). Hence, the value of this field is always between 5 and 15.

- Control flags -

These are 6 1-bit control bits that control connection establishment, connection termination, connection abortion, flow control, mode of transfer etc. Their function is:

URG: Urgent pointer is valid

ACK: Acknowledgement number is valid

PSH: Request for push(push to application layer without waiting in buffer)

RST: Reset the connection(terminate the connection immediately for unrecoverable errors or no normal termination)

SYN: Synchronize sequence numbers

FIN: Terminate the connection

- Window size -

It is 16bits field. This field tells the size of the receiving window of the sender (flow control).

- Checksum -

It is 16 bits field. This field holds the checksum for error control. It is mandatory in TCP as opposed to UDP.

- Urgent pointer -

It is 16bits field. This field (valid only if the URG control flag is set) is used to point how much data in the current segment counting from the first is urgent.

- Options -

There can be up to 40 bytes of optional information in the TCP header. Options field is generally used for timestamp(age of TCP segments helps in differentiating packets with same seq. no), padding(Addition of dummy data(0) to fill up unused space to make standard size(multiple of 4)) etc.

TCP Connection Establishment Using ThreeWay Handshake Process

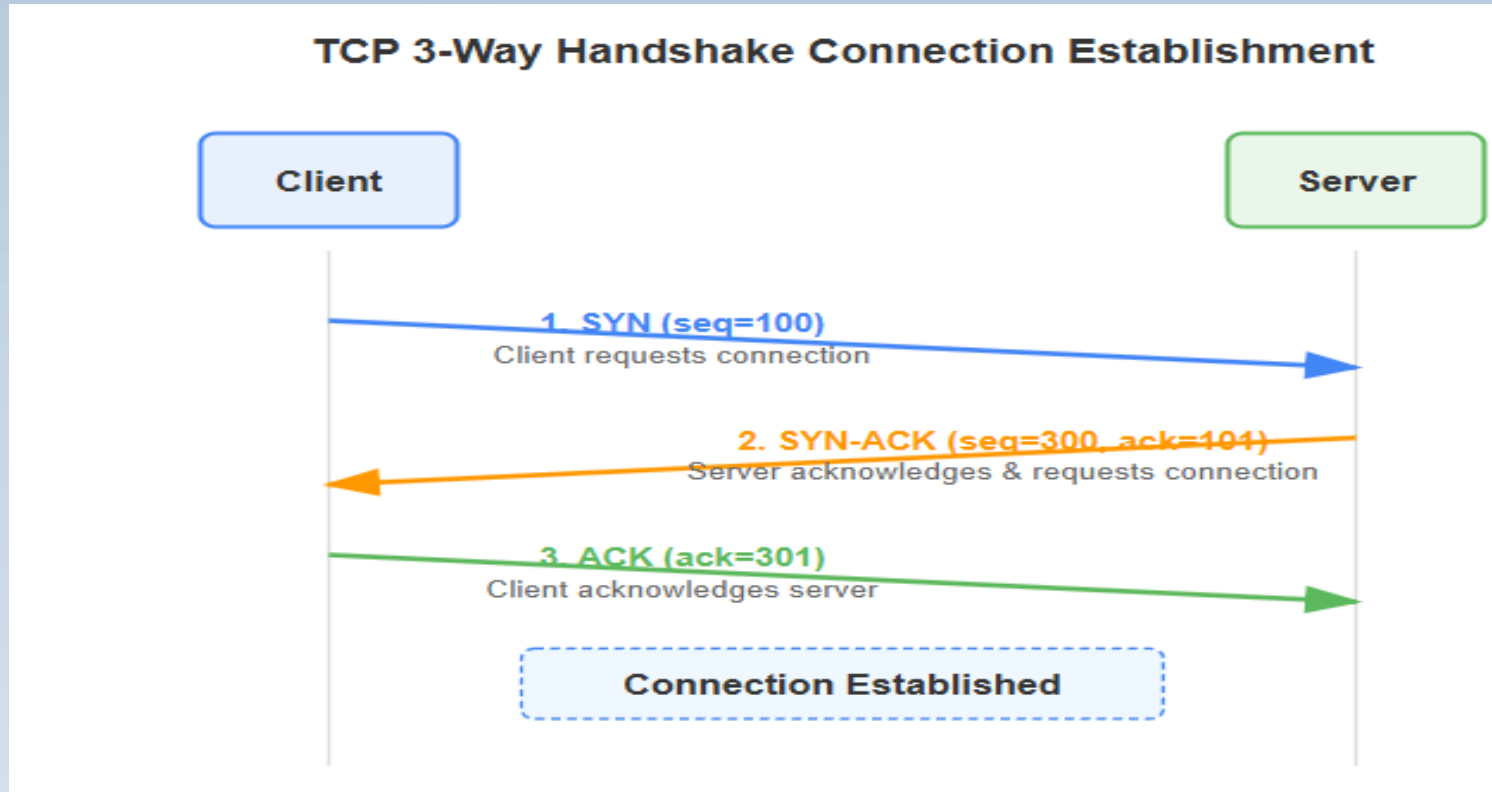
TCP provides a secure and reliable connection between two devices using the 3-way handshake process.

TCP uses a full-duplex connection to synchronize (SYN) and acknowledge (ACK) each other on both sides.

There are three steps for both establishing and closing a connection: SYN, SYN-ACK, and ACK.

The 3-way handshake ensures that both client and server are ready for data transmission and agree on initial sequence numbers for reliable data ordering and flow control.

Connection Establishment Process



Step 1: Synchronization (SYN) – Client Initiates Connection

The client sends a SYN segment to the server with the SYN flag set to 1.

The segment includes an initial sequence number (ISN) – a 32-bit random number used for ordering data segments.

The ACK flag is set to 0, and the client advertises its maximum segment size (MSS) and window size for flow control.

Step 2: Synchronization-Acknowledgment (SYN-ACK) – Server Responds

The server acknowledges the client's SYN by setting the ACK flag to 1 and acknowledgment number to client's sequence number + 1.

The server also sets its SYN flag to 1 with its own initial sequence number if it wants to establish the connection.

The server advertises its window size and maximum segment size to negotiate connection parameters.

Step 3: Acknowledgment (ACK) – Client Confirms
The client sends an ACK segment to acknowledge
the server's SYN.

The acknowledgment number is set to server's sequence number + 1.

Once the server receives this ACK, the connection is fully established and data transmission can begin.

Connection Termination Process

TCP uses a 4-way handshake to properly close connections, ensuring all data is transmitted before termination:

FIN from Client: The client sends a FIN (finish) segment to request connection termination.

ACK from Server: The server acknowledges the FIN request with an ACK segment.

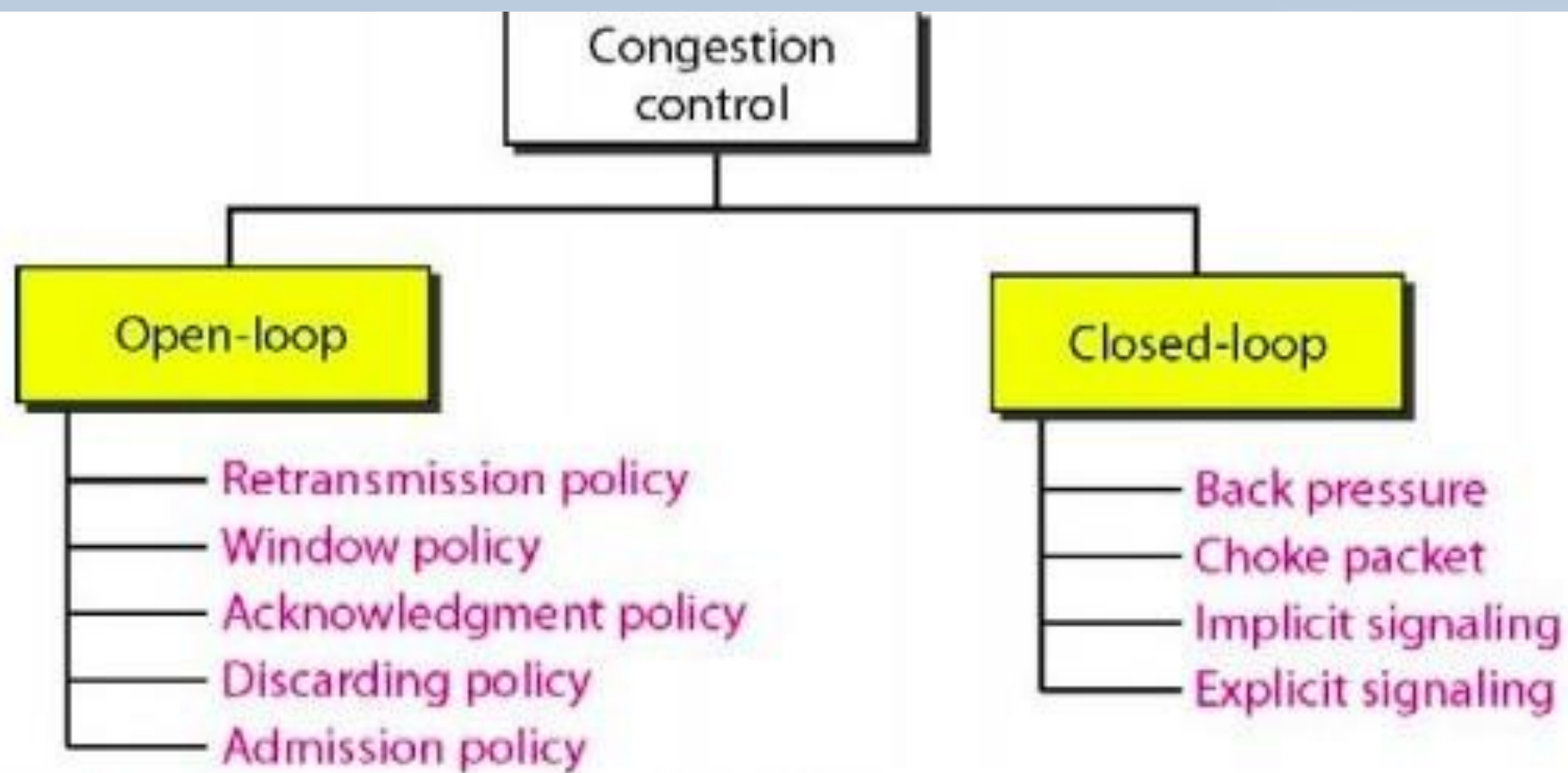
FIN from Server: The server sends its own FIN segment when ready to close its side of the connection.

ACK from Client: The client acknowledges the server's FIN, and the connection is fully closed.

Differentiate between TCP and UDP

5.5 Principle of Congestion Control

- Congestion in a network may occur if load on the network or the no. of packets sent to the network is greater than the capacity of the network.
- Congestion control refers to the techniques used to control or prevent congestion.
- **Congestion control techniques** can be broadly classified into two categories:



1. Open Loop Congestion Control (prevention):

Open loop congestion control policies are applied to prevent congestion before it happens.

Policies adopted by open loop congestion control:

a. Retransmission Policy :

It is the policy in which retransmission of the packets are taken care. If the sender feels that a sent packet is lost or corrupted, the packet needs to be retransmitted. This retransmission may increase the congestion in the network. To prevent congestion, retransmission timers must be designed to prevent congestion.

b. Window Policy :

The type of window at the sender side may also affect the congestion.

Several packets in the Go-back-N Window are resent, although some packets may be received successfully at the receiver side.

This duplication may increase the congestion in the network and making it worse.

Therefore, Selective Repeat Window should be adopted as it sends only the specific packet that may have been lost.

c. Discarding Policy :

A good discarding policy adopted by the routers is that the routers may partially discards the corrupted or less sensitive package and also able to maintain the quality of a message.

d. Acknowledgment Policy :

Since acknowledgments are also the part of the load in network, the acknowledgment policy imposed by the receiver may also affect congestion.

The receiver should send acknowledgement for N packets rather than sending acknowledgement for a single packet.

e. Admission Policy :

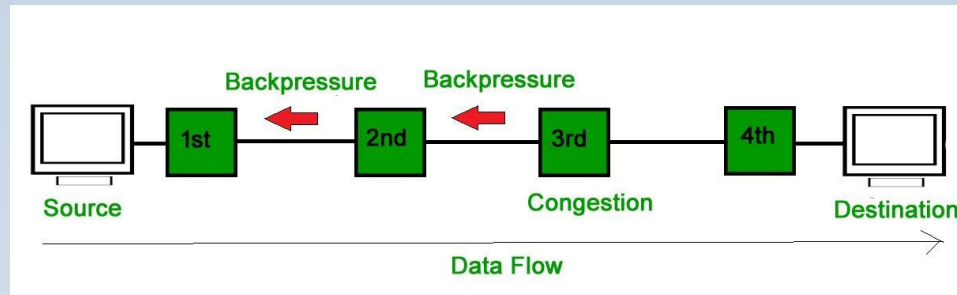
In admission policy a mechanism should be used to prevent congestion. Switches in a flow, first check the resource requirement of a network flow before transmitting it further. If there is a chance of a congestion or there is a congestion in the network, router should deny establishing a virtual network connection to prevent further congestion.

2. Closed Loop Congestion Control(removal):

Closed loop congestion control technique is used to treat congestion after it happens. Several techniques are used; some of them are:

a. Backpressure :

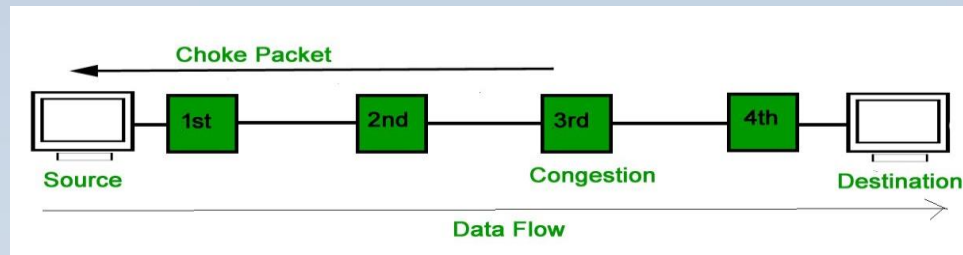
Backpressure is a node-to-node congestion control technique that propagate in the opposite direction of data flow. The backpressure technique can be applied only to virtual circuit where each node has information of its above upstream node.



In above diagram the 3rd node is congested and stops receiving packets as a result 2nd node may get congested due to slowing down of the output data flow. Similarly 1st node may get congested and informs the source to slow down.

b. Choke Packet Technique :

A choke packet is a packet sent by a node to the source to inform it of congestion. Each router monitor its resources and the utilization at each of its output lines. Whenever the resource utilization exceeds the threshold value which is set by the administrator, the router directly sends a choke packet to the source giving it a feedback to reduce the traffic.



c. Implicit Signaling :

In implicit signaling, there is no communication between the congested nodes and the source. The source guesses that there is congestion in a network. For example when sender sends several packets and there is no acknowledgment for a while, one assumption is that there is a congestion.

d. Explicit Signaling :

- In explicit signaling, if a node experiences congestion it can explicitly sends a packet to the source or destination to inform about congestion. The difference between choke packet and explicit signaling is that the signal is included in the packets that carry data rather than creating different packet as in case of choke packet technique.

Explicit signaling can occur in either forward or backward direction.

- Forward Signaling : In forward signaling, signal is sent in the direction of the congestion. The destination is warned about congestion. The receiver in this case adopt policies to prevent further congestion.
- Backward Signaling : In backward signaling, signal is sent in the opposite direction of the congestion. The source is warned about congestion and it needs to slow down.

5.6 Traffic Shaping Algorithms

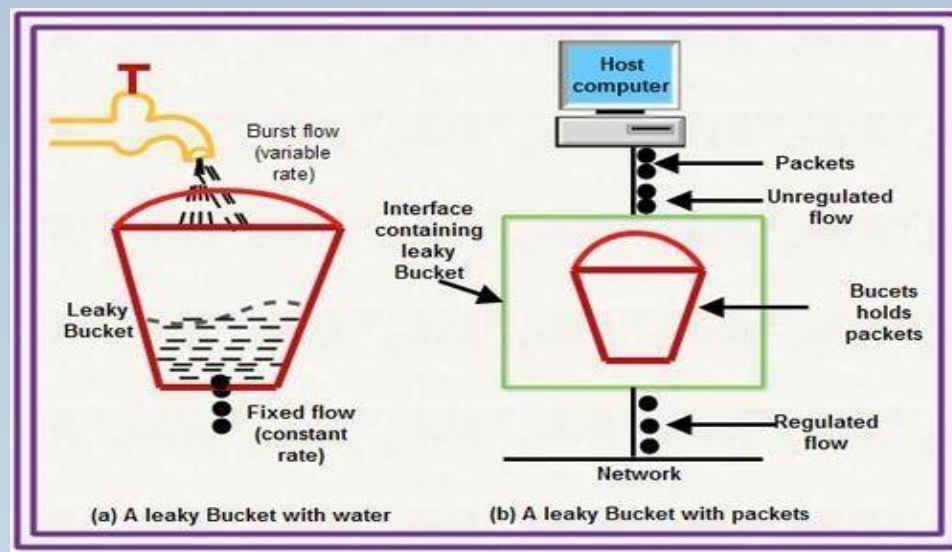
- Traffic Shaping is a mechanism to control the amount and the rate of the traffic sent to the network.
- Approach of congestion management is called Traffic shaping.
- Traffic shaping helps to regulate rate of data transmission and reduces congestion.

There are 2 types of traffic shaping algorithms:

1. Leaky Bucket
2. Token Bucket

1. Leaky Bucket Algorithm:

- It is a traffic shaping mechanism that controls the amount and the rate of the traffic sent to the network.
- A leaky bucket algorithm shapes bursty traffic into fixed rate traffic by averaging the data rate.
- Imagine a bucket with a small hole at the bottom.
- The rate at which the water is poured into the bucket is not fixed and can vary but it leaks from the bucket at a constant rate. Thus (as long as water is present in bucket), the rate at which the water leaks does not depend on the rate at which the water is input to the bucket.
- Also, when the bucket is full, any additional water that enters into the bucket spills over the sides and is lost.



Following steps are involved in leaky bucket algorithm:

- When host wants to send packet, packet is thrown into the bucket.
- The bucket leaks at a constant rate, meaning the network interface transmits packets at a constant rate.
- Bursty traffic is converted to a uniform traffic by the leaky bucket.
- In practice the bucket is a finite queue that outputs at a finite rate.

Algorithm:

Step 1: Initialize the counter to 'n' at every tick of clock

Step 2: If n is greater than the size of packet at the front of queue, send the packet into the network and decrement the counter by size of packet. Repeat the step until 'n' is less than the size of packet.

Step 3: Reset the counter and go to Step 1.

Packet Transmission Cycle

Initialize Counter

Set counter to 'n' at clock tick



Reset Counter

Reset counter for next cycle



Send Packet

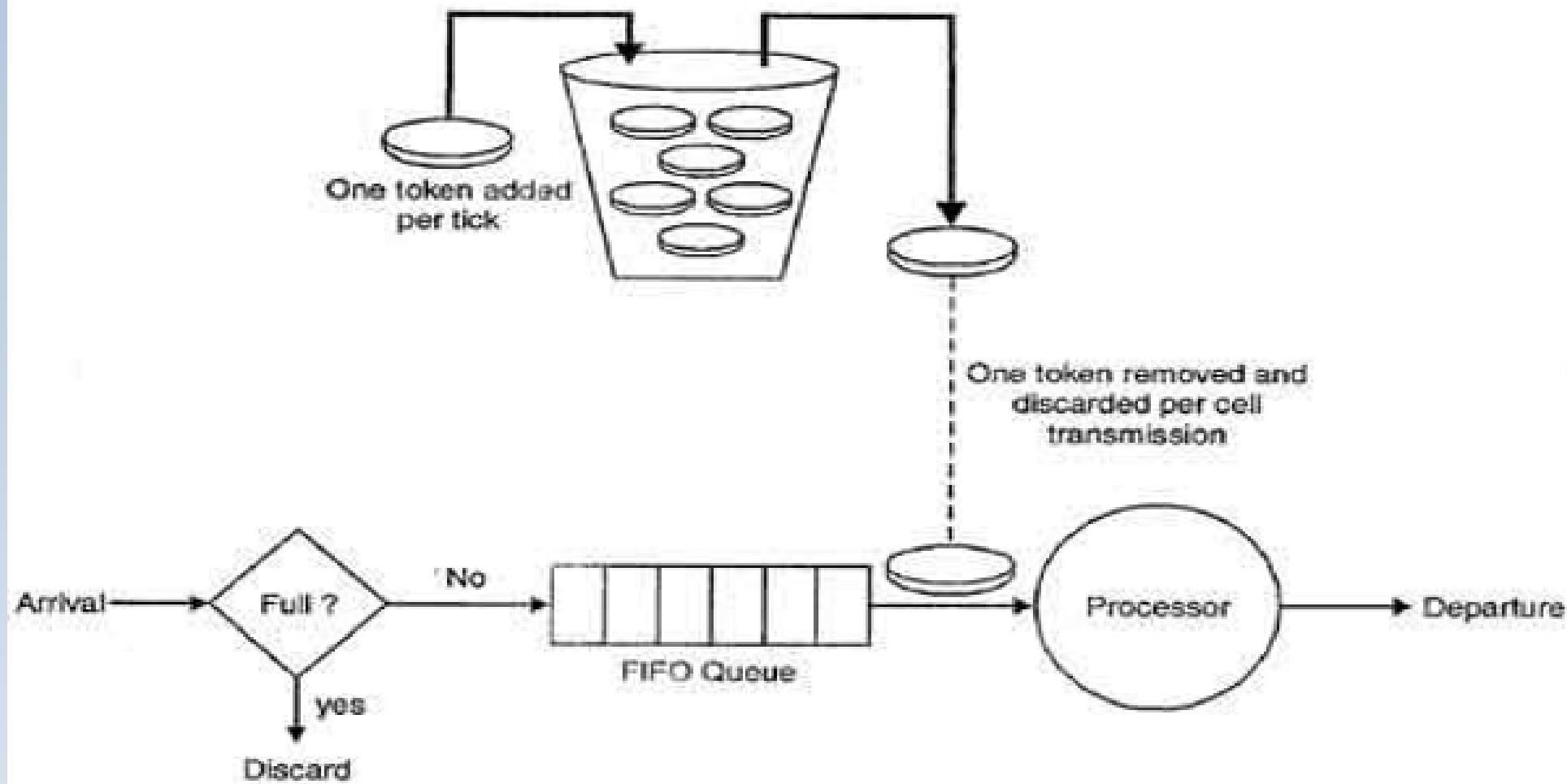
Send packet if counter is sufficient



2. Token Bucket Algorithm:

- The leaky bucket algorithm allows only an average (constant) rate of data flow. Its major problem is that it cannot deal with bursty data.
- A token bucket algorithm allows bursty data transfers.
- A token bucket algorithm is a modification of leaky bucket in which leaky bucket contains tokens.

- In this algorithm, a token(s) are generated at every clock tick. For a packet to be transmitted, system must remove token(s) from the bucket.
- Thus, a token bucket algorithm allows idle hosts to accumulate credit for the future in form of tokens.
- For example, if a system generates 100 tokens in one clock tick and the host is idle for 100 ticks. The bucket will contain 10,000 tokens.
- Now, if the host wants to send bursty data, it can consume all 10,000 tokens at once for sending 10,000 bytes. And if host wanted to send 10,500 bytes then host can send 10,000 bytes and waits 5 clock ticks for sending remaining 500 bytes.
- Thus a host can send bursty data as long as bucket is not empty.



Token bucket algorithm

Algorithm:

- A token is added to the bucket every Δt seconds.
- The bucket can hold at the most b tokens. If a token arrives when the bucket is full, it is discarded.
- When a packet of n bytes arrives,
 - if at least n tokens are in the bucket, n tokens are removed from the bucket, and the packet is sent to the network.
 - if fewer than n tokens are available, no tokens are removed from the bucket, and the packet is considered to be non-compatible.

Some advantage of token bucket over leaky bucket:

- If bucket is full in token bucket, tokens are discarded not packets. While in leaky bucket, packets are discarded.
- Token Bucket can send large bursts at a faster rate while leaky bucket always sends packets at constant rate.

**THANK YOU FOR YOUR
ATTENTION!!!**